CocoERP v2 — Project Roadmap & Audit Playbook

Generated: 2026-01-01

1. Executive summary

- Repository: Google Apps Script ERP built around Google Sheets. Core orchestration lives in **AppCore.js**; feature modules include Purchases, Orders, Logistics/QC, Shipments, Inventory, Catalog, and Sales.
- Automation relies on **installable onEdit** trigger (coco_onEditInstallable) and a **time-driven** trigger (coco_processSyncQueue, typically every 1 minute) to run a staged pipeline.
- Current state (per audits): multiple QC + inventory pipeline functions are referenced from AppCore but missing from the repo, which prevents QC_UAE automation and causes repeated queue re-tries/log spam.
- Primary objective: re-establish end-to-end pipeline correctness first (contract restoration + schema guards), then harden for maintainability (tests, tooling, documentation).

2. System architecture

2.1 Core kernel (AppCore.js)

- Defines canonical constants (APP.SHEETS, APP.COLS, APP.INTERNAL) and shared helpers: header normalization, schema ensuring, safe UI prompts, locking (withLock_), and error logging.
- Owns the single global onOpen(e) and onEdit(e) entrypoints; the installable trigger delegates to module handlers by sheet name.
- Runs coco_processSyncQueue() to process debounced queues/flags stored in DocumentProperties.

2.2 Modules and responsibilities

- **Purchases.js**: purchases layout, defaults/backfill on edit, line IDs, formulas, repair utilities.
- **Orders.js**: Orders summary rebuild and incremental sync from Purchases.
- **Logistics.js**: sheet layouts for QC and shipments, formatting/validations.
- **ShipmentsCore.js**: CN→UAE shipments sync, UAE→EG shipment planning/status, and (expected) QC generation + inventory sync integrations.
- **InventoryCore.js**: inventory ledger writer, snapshot rebuilds (UAE/EG), repair utilities, valuation logic.
- **CatalogCore.js**: catalog sheet layout and sync helpers.
- **Sales.js**: sales sheet layout, onEdit logic, posting sales to inventory ledger.

3. End-to-end data pipeline

Time-driven pipeline (typical order)

- Stage 1 — Purchases → Shipments_CN_UAE: syncPurchasesToShipmentsCnUae().
- Stage 2 — Purchases/Shipments → QC_UAE: qc_generateFromPurchases_(...) (force-all or batched by Order IDs).

- bulletStage 3 — QC_UAE → Inventory ledger (IN UAE) → Inventory_UAE snapshot: syncQCtoInventory_UAE().
- bulletStage 4 — Shipments_UAE_EG → Inventory ledger (OUT UAE, IN EG) → snapshots: syncShipmentsUaeEgToInventory().
- bulletStage 5 — Purchases → Orders sheet: rebuildOrdersSummary() or orders_syncFromPurchasesByOrderIds_(...).

Real-time onEdit handlers (high level)

- bulletPurchases edits: apply defaults, normalize SKU, enqueue Orders/QC, flag CN→UAE shipments sync.
- bulletQC_UAE edits: recalc quantities/result and flag QC→Inventory sync.
- bulletShipments sheets edits: recompute status/totals and flag shipment→inventory sync.
- bulletSales edits: set delivered date and support sales ledger postings.

4. Current blockers and risks

4.1 Blockers (pipeline gaps)

- bullet**Missing**: qc_generateFromPurchases_, qc_generateFromPurchasesPrompt, qcOnEdit_, qc_recalcQuantitiesAndResult, qc_recalcRows_.
- bullet**Missing**: syncQCtoInventory_UAE, shipmentsUaeEgOnEdit_, syncShipmentsUaeEgToInventory.
- bulletImpact: QC_UAE automation stops; queue flags re-set indefinitely; menu items may point to missing handlers.

4.2 High-risk (silent data issues)

- bulletHeader drift and NaN-index reads: several code paths compute map[header] - 1 without asserting header existence; can silently write wrong/empty values.
- bulletPurchases defaults reference keys that may not exist in APP.COLS.PURCHASES (e.g., FX/customs); can result in incomplete defaulting unless literal header names exist.
- bulletSales cost resolution may degrade if Inventory_EG/Catalog headers drift; should be fail-fast with required column asserts.

5. Stabilization plan (phased)

Phase	Goal	Key outputs	Exit criteria
0 — Baseline & freeze	Capture current behavior and [illegible]	Tagged commit + exported reports (lint, syntax, trig[illegible]	[illegible]
1 — Restore c[illegible]	Bring back missing handlers [illegible]	[illegible] QC generation + QC onEdit + QC→In[illegible]	[illegible]
2 — Harden s[illegible]	[illegible]	[illegible] <code>assertRequiredColumns_</code> [illegible]	No silent fallbacks for critical columns; [illegible]
3 — Tooling & tests	Make regressions hard to int[illegible]	Code “contract tests” for header maps and function [illegible]	[illegible]
4 — Feature r[illegible]	Enhance usability and analytics	Dashboards, alerts, cost policy versioning, audit trai[illegible]	[illegible]

6. Collaboration workflow (ChatGPT + Codex + Copilot)

Recommended division of labor

- **Codex (VS Code)**: best for repo-wide scans, multi-file edits, and producing audit/patch artifacts. Use it when you want it to read and modify many files safely.
- **Copilot Chat (VS Code)**: best for interactive edits in the file you are currently in, small diffs, and quick refactors.
- **ChatGPT (web)**: best as the “architect/PM” layer: roadmap, design decisions, verifying invariants, reviewing outputs from Codex/Copilot, and deciding what to merge.

Standard loop per task

1. 1) Define scope + acceptance criteria (one paragraph + checklist).
2. 2) Ask Codex for read-only scan & proposal (no changes) OR for a targeted patch on a new branch.
3. 3) Ask Copilot for alternative patch/diff (small, local).
4. 4) Compare proposals, choose best or merge, then run npm run check:syntax and npm run lint.
5. 5) Deploy via clasp push, validate in Apps Script (manual run + trigger run), then commit.

7. Operational checklists

Pre-change safety checklist

- Confirm current triggers: installable onEdit + time-based queue runner.
- Export DocumentProperties keys for queues/flags (for backup).
- Run local: npm run check:syntax and npm run lint (warnings are acceptable; errors are not).
- Use a dedicated branch per fix and keep diffs small and reversible.

Post-change verification

- Manual: run coco_processSyncQueueNow() and confirm no errors.
- Functional: edit Purchases/QC/Shipments and verify corresponding flags are set and then cleared by the queue runner.
- Data: verify ledger idempotency (no duplicate Txn IDs) and snapshots match ledger totals.